

React.js Fundamentals

Course Information

Course Name: React.js Fundamentals

Level: Beginner to Intermediate

Prerequisites: Basic HTML, CSS, and JavaScript

Estimated Duration: 12–15 Hours

Course Description

React.js is a JavaScript library used to build interactive and component-based user interfaces. In this course, students will learn the fundamentals of React, including components, JSX, props, state, events, conditional rendering, lists, forms, hooks, and API integration.

Students will gradually move from basic React concepts to building a complete interactive web application. By the end of the course, students will be able to create reusable React components and develop a small frontend application.

Learning Outcomes

After completing this course, students will be able to:

- Explain what React.js is and why it is used.
- Understand the basic structure of a React application.
- Create React components.
- Use JSX to define user interfaces.
- Pass data between components using props.
- Manage component data using state.
- Handle user events.
- Perform conditional rendering.
- Render lists using JavaScript.
- Build controlled forms.
- Understand React Hooks.
- Use the `useState` Hook.
- Use the `useEffect` Hook.
- Fetch data from an API.
- Organize a React application into reusable components.
- Build a complete small React project.

Lesson 1: Introduction to React.js

Learning Objectives

After this lesson, students will be able to:

- Define React.js.
- Explain why React is used.
- Understand component-based development.
- Identify the role of React in modern web development.

What is React?

React is a JavaScript library for building user interfaces.

React allows developers to divide an interface into small reusable components.

For example, a webpage may contain:

- Header
- Navigation
- Sidebar
- Product Card
- Footer

Each part can be created as a separate React component.

React and JavaScript

React is built using JavaScript.

A basic JavaScript statement:

```
const name = "Ali";  
  
console.log(name);
```

React uses JavaScript together with JSX to create user interfaces.

Why Use React?

React provides several useful features:

- Component-based development
 - Reusable UI components
 - State management
 - Event handling
 - Efficient UI updates
 - Support for large applications
-

React Application Example

A simple React component can look like:

```
function Welcome() {  
  return <h1>Hello World</h1>;  
}
```

The component can then be displayed inside an application.

Lesson 2: React Project Structure

Learning Objectives

After this lesson, students will be able to:

- Understand the structure of a React project.
- Identify important files.
- Understand the role of components.

Creating a React Application

A React project can be created using modern development tools.

A typical project may contain:

```
my-react-app/  
|  
├─ public/  
├─ src/
```

```
|   |   | App.jsx  
|   |   | main.jsx  
|   |   | components/  
|   |  
|   | package.json  
|   | index.html
```

The src Folder

The `src` folder normally contains application source code.

For example:

```
src/  
|   | App.jsx  
|   | main.jsx  
|   | components/
```

App.jsx

`App.jsx` commonly contains the main application component.

Example:

```
function App() {  
  return (  
    <h1>My React Application</h1>  
  );  
}  
  
export default App;
```

main.jsx

The entry point connects the React application to the webpage.

Example:

```
import React from "react";
import ReactDOM from "react-dom/client";
import App from "./App.jsx";

ReactDOM.createRoot(
  document.getElementById("root")
).render(
  <App />
);
```

Lesson 3: Components

React Components

Components are independent and reusable parts of a user interface.

Example:

```
function Header() {
  return (
    <header>
      <h1>My Website</h1>
    </header>
  );
}
```

Using a Component

A component can be used inside another component.

```
function App() {
  return (
    <div>
      <Header />
      <h2>Welcome</h2>
    </div>
  );
}
```

Reusable Components

A component can be reused multiple times.

```
function Button() {  
  return <button>Click Me</button>;  
}  
  
function App() {  
  return (  
    <div>  
      <Button />  
      <Button />  
      <Button />  
    </div>  
  );  
}
```

Component Naming

React components normally use PascalCase.

Correct:

```
function StudentCard() {  
  return <div>Student</div>;  
}
```

Lesson 4: JSX

What is JSX?

JSX is a syntax extension that allows developers to write HTML-like markup inside JavaScript.

Example:

```
function App() {  
  return (  
    <div>
```

```
        <h1>Hello</h1>
        <p>Welcome to React.</p>
      </div>
    );
  }
}
```

JavaScript Inside JSX

JavaScript expressions can be written using curly braces.

```
function App() {
  const name = "Ali";

  return (
    <h1>Hello {name}</h1>
  );
}
```

Output:

```
Hello Ali
```

JSX Attributes

JSX uses attributes similar to HTML.

```

```

Some attribute names differ from normal HTML.

For example:

```
<div className="container">
  Content
</div>
```

JSX Expressions

Example:

```
function App() {

  const age = 20;

  return (
    <p>
      Age: {age}
    </p>
  );
}
```

Lesson 5: Props

What are Props?

Props are used to pass data from one component to another.

Example:

```
function Student(props) {
  return (
    <h2>
      Student: {props.name}
    </h2>
  );
}
```

Passing Props

```
function App() {  
  return (  
    <Student name="Ali" />  
  );  
}
```

The `name` value is passed to the `Student` component.

Multiple Props

```
function Student(props) {  
  return (  
    <div>  
      <h2>{props.name}</h2>  
      <p>Age: {props.age}</p>  
      <p>Course: {props.course}</p>  
    </div>  
  );  
}
```

Usage:

```
<Student  
  name="Ali"  
  age={21}  
  course="Computer Science"  
/>
```

Destructuring Props

Props can also be destructured.

```
function Student({ name, age }) {  
  return (  
    <div>  
      <h2>{name}</h2>  
      <p>{age}</p>  
    </div>  
  );  
}
```

```
    </div>
  );
}
```

Lesson 6: Events

Handling Events

React allows components to respond to user actions.

Examples include:

- Clicking buttons
- Typing in inputs
- Submitting forms
- Moving the mouse

Example:

```
function App() {

  function handleClick() {
    alert("Button clicked!");
  }

  return (
    <button onClick={handleClick}>
      Click Me
    </button>
  );
}
```

Inline Event Handling

```
<button
  onClick={() => alert("Hello")}
>
  Click
</button>
```

Input Events

```
function App() {  
  
  function handleChange(event) {  
    console.log(event.target.value);  
  }  
  
  return (  
    <input  
      type="text"  
      onChange={handleChange}  
    />  
  );  
}
```

Lesson 7: State and useState

What is State?

State represents data that can change during the lifetime of a component.

Examples include:

- Counter value
- Username
- Login status
- Selected item
- Form data

useState

React provides the `useState` Hook for managing state.

```
import { useState } from "react";  
  
function Counter() {  
  
  const [count, setCount] = useState(0);  
  
  return (  

```

```
    <div>
      <p>Count: {count}</p>

      <button
        onClick={() => setCount(count + 1)}
      >
        Increase
      </button>
    </div>
  );
}
```

Updating State

The setter function updates state.

```
setCount(count + 1);
```

React then updates the component's user interface.

Multiple State Variables

```
function App() {

  const [name, setName] = useState("");
  const [age, setAge] = useState(0);

  return (
    <div>
      <p>Name: {name}</p>
      <p>Age: {age}</p>
    </div>
  );
}
```

Lesson 8: Conditional Rendering

What is Conditional Rendering?

Conditional rendering means displaying different content based on a condition.

Example:

```
function App() {  
  
  const isLoggedIn = true;  
  
  return (  
    <div>  
      {isLoggedIn  
        ? <h1>Welcome</h1>  
        : <h1>Please Login</h1>  
      }  
    </div>  
  );  
}
```

Using Logical AND

```
function App() {  
  
  const isAdmin = true;  
  
  return (  
    <div>  
      {isAdmin && (  
        <p>Admin Panel</p>  
      )}  
    </div>  
  );  
}
```

Conditional Components

```
function Status({ online }) {  
  
  if (online) {  
    return <p>User is online</p>;  
  }  
  
  return <p>User is offline</p>;  
}
```

Lesson 9: Lists and Keys

Rendering Lists

JavaScript's `map()` method can be used to display lists.

```
function App() {  
  
  const students = [  
    "Ali",  
    "Ahmed",  
    "Sara"  
  ];  
  
  return (  
    <ul>  
      {students.map((student) => (  
        <li>{student}</li>  
      ))}  
    </ul>  
  );  
}
```

Keys

React uses keys to identify elements in a list.

```
const students = [
  { id: 1, name: "Ali" },
  { id: 2, name: "Ahmed" },
  { id: 3, name: "Sara" }
];

function App() {

  return (
    <ul>
      {students.map((student) => (
        <li key={student.id}>
          {student.name}
        </li>
      ))}
    </ul>
  );
}
```

Why Keys Are Important

Keys help React identify which list items have changed, been added, or removed.

Lesson 10: Forms

React Forms

React can manage form inputs using state.

Example:

```
import { useState } from "react";

function Form() {

  const [name, setName] = useState("");

  return (
    <form>
      <input
        type="text"

```

```

        value={name}
        onChange={(event) =>
            setName(event.target.value)
        }
    />

    <button type="submit">
        Submit
    </button>
</form>
);
}

```

Controlled Components

An input whose value is controlled by React state is called a controlled component.

Example:

```

const [email, setEmail] = useState("");

<input
  type="email"
  value={email}
  onChange={(event) =>
    setEmail(event.target.value)
  }
/>

```

Multiple Form Fields

```

function Registration() {

  const [name, setName] = useState("");
  const [email, setEmail] = useState("");

  return (
    <form>
      <input
        value={name}
        onChange={(e) =>

```

```

        setName(e.target.value)
      }
    />

    <input
      value={email}
      onChange={(e) =>
        setEmail(e.target.value)
      }
    />
  </form>
);
}

```

Lesson 11: useEffect Hook

What is useEffect?

`useEffect` allows a component to perform side effects.

Examples include:

- Fetching data
- Updating the document title
- Working with external systems
- Running code after rendering

Example:

```

import {
  useEffect
} from "react";

function App() {

  useEffect(() => {
    console.log("Component loaded");
  }, []);

  return (
    <h1>Hello React</h1>

```

```
    );  
  }
```

Dependency Array

The dependency array controls when an effect runs.

```
useEffect(() => {  
    console.log("Effect executed");  
}, []);
```

An empty dependency array commonly means the effect runs after the initial render.

Effect With State

```
const [count, setCount] = useState(0);  
  
useEffect(() => {  
    document.title =  
        `Count: ${count}`;  
}, [count]);
```

Lesson 12: Fetching Data from an API

What is an API?

An API allows applications to communicate with other software systems and retrieve or send data.

React applications can use JavaScript's `fetch()` function to request data.

Basic Fetch Request

```
fetch("https://example.com/api/students")
  .then(response => response.json())
  .then(data => {
    console.log(data);
  });
```

Fetching Data With useEffect

```
import {
  useEffect,
  useState
} from "react";

function Students() {

  const [students, setStudents] =
    useState([]);

  useEffect(() => {

    fetch(
      "https://example.com/api/students"
    )
      .then(response => response.json())
      .then(data => {
        setStudents(data);
      });

  }, []);

  return (
    <div>
      {students.map(student => (
        <p key={student.id}>
          {student.name}
        </p>
      ))}
    </div>
  );
}
```

Loading State

Applications can display a loading message while waiting for data.

```
const [loading, setLoading] =  
  useState(true);
```

Example:

```
{loading  
  ? <p>Loading...</p>  
  : <StudentList />  
}
```

Lesson 13: Component Communication

Parent to Child

Props allow a parent component to send data to a child component.

```
function Child({ message }) {  
  
  return (  
    <p>{message}</p>  
  );  
}  
  
function Parent() {  
  
  return (  
    <Child message="Hello Child" />  
  );  
}
```

Child to Parent

A parent can pass a function to a child.

```
function Child({ onMessage }) {

  return (
    <button
      onClick={() =>
        onMessage("Hello Parent")
      }
    >
      Send Message
    </button>
  );
}
```

Parent:

```
function Parent() {

  function receiveMessage(message) {
    console.log(message);
  }

  return (
    <Child
      onMessage={receiveMessage}
    />
  );
}
```

Lesson 14: React Application Organization

Organizing Components

A larger React application can separate components into different files.

Example:

```
src/
|
├── components/
|   ├── Header.jsx
|   └── Footer.jsx
```

```
|   |   | Navbar.jsx
|   |   | StudentCard.jsx
|
|   | pages/
|   |   | Home.jsx
|   |   | About.jsx
|   |   | Contact.jsx
|
|   | App.jsx
|   | main.jsx
```

Exporting a Component

```
function Header() {

    return (
        <header>
            <h1>My Website</h1>
        </header>
    );
}

export default Header;
```

Importing a Component

```
import Header
    from "../components/Header";

function App() {

    return (
        <div>
            <Header />
        </div>
    );
}
```

Lesson 15: Complete React Application

The following example combines components, props, state, events, lists, and forms.

```
import { useState } from "react";

function StudentCard({ name, course }) {

  return (
    <div>
      <h3>{name}</h3>
      <p>Course: {course}</p>
    </div>
  );
}

function App() {

  const [name, setName] = useState("");

  const students = [
    {
      id: 1,
      name: "Ali",
      course: "Web Development"
    },
    {
      id: 2,
      name: "Ahmed",
      course: "Computer Science"
    },
    {
      id: 3,
      name: "Sara",
      course: "Data Science"
    }
  ];

  return (
    <div>

      <h1>Student Dashboard</h1>

      <input
        type="text"
        placeholder="Enter your name"
      />
    </div>
  );
}
```

```

        value={name}
        onChange={(event) =>
            setName(event.target.value)
        }
    />

    <p>
        Welcome {name}
    </p>

    <h2>Students</h2>

    {students.map(student => (
        <StudentCard
            key={student.id}
            name={student.name}
            course={student.course}
        />
    ))}

    </div>
    );
}

export default App;

```

Mini Project

Student Management Dashboard

Create a React-based student management dashboard.

Requirements

The application should contain:

1. Header
2. Navigation
3. Student list
4. Student cards
5. Search input
6. Student registration form
7. Course information
8. Student count

9. Footer

Functional Requirements

The application should allow users to:

- View students.
- Search for students.
- Add a new student.
- Display student information.
- Display the number of students.
- Submit a form.
- Dynamically update the interface.

React Concepts to Use

The project should demonstrate:

- React components
- JSX
- Props
- State
- Events
- Conditional rendering
- Lists
- Keys
- Forms
- `useState`
- `useEffect`
- API data

Practice Exercises

Exercise 1

Create a React component that displays:

- Student name
- Student age
- Student course

Use props to provide the information.

Exercise 2

Create a counter application containing:

- Current count
- Increase button
- Decrease button
- Reset button

Use `useState`.

Exercise 3

Create a login interface containing:

- Username
- Password
- Login button

Display a welcome message after the login state changes.

Exercise 4

Create a student list using an array and `map()`.

Each student should contain:

- ID
- Name
- Course
- Age

Use the student's ID as the React key.

Exercise 5

Create a search application that allows users to search a list of students.

The interface should update according to the search text.

Exercise 6

Create a form that accepts:

- Name
- Email
- Course
- Phone number

Display the submitted information on the page.

Final Quiz

1. What is React.js?
 2. Why is React used for frontend development?
 3. What is a React component?
 4. What is JSX?
 5. How is JSX different from normal HTML?
 6. What are props?
 7. How can props be passed to a component?
 8. What is state in React?
 9. What is the purpose of `useState` ?
 10. What is event handling in React?
 11. What is conditional rendering?
 12. Why does React use keys when rendering lists?
 13. What is a controlled component?
 14. What is the purpose of `useEffect` ?
 15. What is an API?
 16. How can React fetch data from an API?
 17. How can a parent component communicate with a child component?
 18. How can a child component send information to a parent?
 19. Why should React applications be divided into reusable components?
 20. What is the purpose of a React project structure?
-

Course Summary

After completing this course, students should understand the fundamentals of React.js and be able to create interactive component-based web applications.

The major concepts covered are:

- React introduction
- React project structure

- Components
- JSX
- Props
- Events
- State
- `useState`
- Conditional rendering
- Lists
- Keys
- Forms
- Controlled components
- `useEffect`
- API integration
- Component communication
- Application organization
- Complete React application development

Next Step

After completing React.js Fundamentals, students can continue toward more advanced frontend development topics such as:

- React Router
- Advanced Hooks
- Context API
- State management
- Authentication
- REST APIs
- Full-stack development
- Node.js and Express
- Database integration
- Deployment

This course follows the same **course → lessons → concepts → examples → exercises → mini-project → quiz → summary** structure as the provided HTML course, making it suitable for maintaining consistency across your Web Development course collection.